

Cloud design patterns

Architects design workloads by integrating platform services, functionality, and code to meet both functional and nonfunctional requirements. To design effective workloads, you must understand these requirements and select topologies and methodologies that address the challenges of your workload's constraints. Cloud design patterns provide solutions to many common challenges.

System design heavily relies on established design patterns. You can design infrastructure, code, and distributed systems by using a combination of these patterns. These patterns are crucial for building reliable, highly secure, cost-optimized, operationally efficient, and high-performing applications in the cloud.

The following cloud design patterns are technology-agnostic, which makes them suitable for any distributed system. You can apply these patterns across Azure, other cloud platforms, on-premises setups, and hybrid environments.

How cloud design patterns enhance the design process

Cloud workloads are vulnerable to the [fallacies of distributed computing](#), which are common but incorrect assumptions about how distributed systems operate. Examples of these fallacies include:

- The network is reliable.
- Latency is zero.
- Bandwidth is infinite.
- The network is secure.
- Topology doesn't change.
- There's one administrator.
- Component versioning is simple.
- Observability implementation can be delayed.

These misconceptions can result in flawed workload designs. Design patterns don't eliminate these misconceptions but help raise awareness, provide compensation strategies, and provide mitigations. Each cloud design pattern has trade-offs. Focus on why you should choose a specific pattern instead of how to implement it.

Consider how to use these industry-standard design patterns as the core building blocks for a well-architected workload design. Each design pattern in the Azure Well-Architected Framework represents one or more of its pillars. Some patterns might introduce trade-offs that affect the goals of other pillars.

Pattern catalog

Each pattern in this catalog describes the problem that it addresses, considerations for applying the pattern, and an example based on Microsoft Azure services and tools. Some patterns include code samples or snippets that show how to implement the pattern on Azure.

[Expand table](#)

Pattern	Summary	Well-Architected Framework pillars
Ambassador	Create helper services that send network requests on behalf of a consumer service or application.	- Reliability - Security
Anti-Corruption Layer	Implement a façade or adapter layer between a modern application and a legacy system.	- Operational Excellence
Asynchronous Request-Reply	Decouple back-end processing from a front-end host. This pattern is useful when back-end processing must be asynchronous, but the front end requires a clear and timely response.	- Performance Efficiency
Backends for Frontends	Create separate backend services for specific frontend applications or interfaces.	- Reliability - Security - Performance Efficiency
Bulkhead	Isolate elements of an application into pools so that if one fails, the others continue to function.	- Reliability - Security - Performance Efficiency
Cache-Aside	Load data on demand into a cache from a data store.	- Reliability - Performance Efficiency
Choreography	Let individual services decide when and how a business operation is processed, instead of depending on a central orchestrator.	- Operational Excellence - Performance Efficiency
Circuit Breaker	Handle faults that might take a variable amount of time to fix when an application connects to a remote service or resource.	- Reliability - Performance Efficiency
Claim Check	Split a large message into a claim check and a payload to avoid overwhelming a message bus.	- Reliability - Security

Pattern	Summary	Well-Architected Framework pillars
		- Cost Optimization - Performance Efficiency
Compensating Transaction	Undo the work performed by a sequence of steps that collectively form an eventually consistent operation.	- Reliability
Competing Consumers	Enable multiple concurrent consumers to process messages that they receive on the same messaging channel.	- Reliability - Cost Optimization - Performance Efficiency
Compute Resource Consolidation	Consolidate multiple tasks or operations into a single computational unit.	- Cost Optimization - Operational Excellence - Performance Efficiency
CQRS	Separate operations that read data from those that update data by using distinct interfaces.	- Performance Efficiency
Deployment Stamps	Deploy multiple independent copies of application components, including data stores.	- Operational Excellence - Performance Efficiency
Event Sourcing	Use an append-only store to record a full series of events that describe actions taken on data in a domain.	- Reliability - Performance Efficiency
External Configuration Store	Move configuration information out of an application deployment package to a centralized location.	- Operational Excellence
Federated Identity	Delegate authentication to an external identity provider.	- Reliability - Security - Performance Efficiency
Gateway Aggregation	Use a gateway to aggregate multiple individual requests into a single request.	- Reliability - Security - Operational Excellence

Pattern	Summary	Well-Architected Framework pillars
		- Performance Efficiency
Gateway Offloading	Offload shared or specialized service functionality to a gateway proxy.	- Reliability - Security - Cost Optimization - Operational Excellence - Performance Efficiency
Gateway Routing	Route requests to multiple services by using a single endpoint.	- Reliability - Operational Excellence - Performance Efficiency
Geode	Deploy back-end services across geographically distributed nodes. Each node can handle client requests from any region.	- Reliability - Performance Efficiency
Health Endpoint Monitoring	Implement functional checks in an application that external tools can access through exposed endpoints at regular intervals.	- Reliability - Operational Excellence - Performance Efficiency
Index Table	Create indexes over the fields in data stores that queries frequently reference.	- Reliability - Performance Efficiency
Leader Election	Coordinate actions in a distributed application by electing one instance as the leader. The leader manages a collection of collaborating task instances.	- Reliability
Materialized View	Generate prepopulated views over the data in one or more data stores when the data is poorly formatted for required query operations.	- Performance Efficiency
Messaging Bridge	Build an intermediary to enable communication between messaging systems that are otherwise incompatible.	- Cost Optimization - Operational Excellence
Pipes and Filters	Break down a task that performs complex processing into a series of separate elements that	- Reliability

Pattern	Summary	Well-Architected Framework pillars
	can be reused.	
Priority Queue	Prioritize requests sent to services so that requests with a higher priority are processed more quickly.	- Reliability - Performance Efficiency
Publisher/Subscriber	Enable an application to announce events to multiple consumers asynchronously, without coupling senders to receivers.	- Reliability - Security - Cost Optimization - Operational Excellence - Performance Efficiency
Quarantine	Ensure that external assets meet a team-agreed quality level before the workload consumes them.	- Security - Operational Excellence
Queue-Based Load Leveling	Use a queue that creates a buffer between a task and a service to smooth intermittent heavy loads.	- Reliability - Cost Optimization - Performance Efficiency
Rate Limiting	Avoid or minimize throttling errors by controlling the consumption of resources.	- Reliability
Retry	Enable applications to handle anticipated temporary failures by retrying failed operations.	- Reliability
Saga	Manage data consistency across microservices in distributed transaction scenarios.	- Reliability
Scheduler Agent Supervisor	Coordinate a set of actions across distributed services and resources.	- Reliability - Performance Efficiency
Sequential Convoy	Process a set of related messages in a defined order without blocking other message groups.	- Reliability
Sharding	Divide a data store into a set of horizontal partitions or shards.	- Reliability - Cost Optimization

Pattern	Summary	Well-Architected Framework pillars
Sidecar	Deploy components into a separate process or container to provide isolation and encapsulation.	- Security - Operational Excellence
Static Content Hosting	Deploy static content to a cloud-based storage service for direct client delivery.	- Cost Optimization
Strangler Fig	Incrementally migrate a legacy system by gradually replacing pieces of functionality with new applications and services.	- Reliability - Cost Optimization - Operational Excellence
Throttling	Control the consumption of resources from applications, tenants, or services.	- Reliability - Security - Cost Optimization - Performance Efficiency
Valet Key	Use a token or key to provide clients with restricted, direct access to a specific resource or service.	- Security - Cost Optimization - Performance Efficiency

AI agent orchestration patterns

The preceding cloud design patterns address common challenges in distributed systems, but AI workloads that use multiple autonomous agents require specialized coordination approaches. Traditional patterns like Scheduler Agent Supervisor or Choreography provide foundational concepts. However, AI agents introduce unique challenges such as nondeterministic outputs, dynamic reasoning capabilities, and the need for intelligent handoffs between specialized components.

For AI workloads that include multiple autonomous agents, see [AI agent orchestration patterns](#). These patterns complement the cloud design patterns in this catalog by addressing the specific coordination requirements of intelligent, autonomous components that work together to accomplish complex outcomes.

Next steps

Review the design patterns from the perspective of the Well-Architected Framework pillar that the pattern aims to optimize.

- [Design patterns to support the Reliability pillar](#)
- [Design patterns to support the Security pillar](#)
- [Design patterns to support the Cost Optimization pillar](#)
- [Design patterns to support the Operational Excellence pillar](#)
- [Design patterns to support the Performance Efficiency pillar](#)

Last updated on 07/19/2025